

Introduction to R & Environmental Data

SHM 377 Hazardous Materials Management

Yoni Rodriguez

Table of contents

Virtual Lab 01	3
Introduction to R & Environmental Data	3
The Monitoring Question	4
The Dataset	4
Why Are We Using R?	5
R and RStudio	5
What is R?	5
What is RStudio?	6
Installing R and RStudio	6
Windows	7
macOS	7
Check Your Installation	7
The RStudio Interface	8
Organizing Your Work	9
Create a Course Folder	9
File Naming	9
Creating Your First R Script	10
Open a New R Script	10
Save Your R Script	10
Set the Working Directory	11
Your First R Commands	11
Creating an Object	12
Getting the Dataset	13
Download the Dataset	13

Importing the Dataset	13
Reading R Code	14
Commands and Functions	14
Referring to a Variable	15
Inspect Before You Analyze	16
Look at the First Rows	16
Look at the Last Rows	17
Variable Names	17
Dataset Dimensions	18
Examine the Structure	19
Did R Understand the Date and Time?	19
Check the Measurements	21
Minimum	21
Maximum	22
Unusual Values	22
Describe the Data	23
Mean	23
Median	23
Minimum	24
Maximum	24
Summary	24
Descriptive Statistics Have Limits	25
Visualizing Concentration Through Time	25
Reading the Plot Function	26
Arguments	27
Why Do We Use Commas?	27
Why Does <code>date_time</code> Come First?	28
Positional Arguments	28
Named Arguments	29
What Does <code>type = "l"</code> Mean?	29
What Does <code>xlab</code> Mean?	30
What Does <code>ylab</code> Mean?	30
What Does <code>= Mean</code> Inside the Function?	30
Read the Entire Command	31
Why Units Matter	32

Look Before Explaining	33
Observation and Interpretation	33
What Can We Conclude?	34
What Can We Not Conclude?	34
What Other Data Could Help?	34
Return to the Original Question	34
The SHM 377 Data Workflow	35
Connecting Data to Hazardous Materials Management	35
Reproducibility	36
You Do Not Need to Memorize Everything	37
What You Accomplished	37
The Main Idea	38
Where We Go Next	38

Virtual Lab 01

Introduction to R & Environmental Data

Today we will use a real environmental monitoring dataset to answer a simple question:

What happened to ambient particle concentrations during five days of winter monitoring?

We will use R to:

- obtain the data
- inspect the data
- check the measurements
- calculate simple summaries
- create a visualization
- interpret what we see

The goal is not to become a programmer today.

The goal is to begin using R as a tool for environmental and hazardous materials data analysis.

The Monitoring Question

We have repeated measurements of airborne particles collected during winter monitoring.

The measurements were collected approximately every two minutes from:

December 27 through December 31, 2019

Our question is:

What happened to ambient particle concentrations during these five days?

Before doing any analysis, we need to understand what the dataset contains.

The Dataset

The teaching dataset contains approximately 3,600 observations.

It contains two variables:

Variable	Meaning
<code>date_time</code>	When the measurement was collected
<code>concentration</code>	Measured particle concentration

The measurements were collected using a **TSI DustTrak**.

Particle concentration was measured in **milligrams per cubic meter (mg/m³)**.

The monitoring instrument measured particle concentration.

It did not identify the exact source of every particle.

This distinction is important.

Measurement does not equal source attribution.

If concentration increases, we can observe the increase.

We cannot automatically conclude what caused it.

Why Are We Using R?

Environmental and hazardous materials work often produces large amounts of data.

Examples include:

- air monitoring
- water sampling
- soil sampling
- surface sampling
- direct reading instrument data
- meteorological measurements
- exposure measurements

R gives us a way to organize, inspect, analyze, visualize, and communicate those data.

Another major advantage is reproducibility.

Instead of clicking through a series of menus and trying to remember what we did, we can save the commands used for the analysis.

R and RStudio

We will use two pieces of software in this course: **R** and **RStudio**.

They work together, but they are not the same thing.

R is the program that performs the analysis. RStudio is the interface we use to work with R.

Both are available for free.

What is R?



R is a programming language and analytical environment used for:

- data analysis
- statistics
- visualization
- reproducible research

When we ask R to calculate a mean, summarize a dataset, or create a graph, **R is the software actually performing those operations.**

R is free software and is available for Windows, macOS, and Linux.

[Download R from the R Project](#)

What is RStudio?



RStudio is an integrated development environment, or IDE, that provides a convenient interface for working with R.

Instead of working with R through a basic command window, RStudio gives us an organized workspace where we can:

- write and save code
- run R commands
- view our data
- create and view graphs
- navigate files
- access help and documentation

RStudio Desktop is available in a free open source edition.

[Download RStudio Desktop from Posit](#)

Installing R and RStudio

Before we begin working with data, we need both **R** and **RStudio Desktop** installed on your computer.

Both are free.

! Installation order matters

Install the software in this order:

1. **Install R**
2. **Install RStudio Desktop**
3. **Open RStudio**
4. **Confirm that RStudio can find and run R**

RStudio provides the interface, but R performs the computations.

Windows

1. Go to the [R Project](#).
2. Select **CRAN**.
3. Choose a CRAN mirror.
4. Select **Download R for Windows**.
5. Download the current version of R.
6. Run the installer and follow the installation prompts.
7. Go to the [RStudio Desktop download page](#).
8. Download the free version of RStudio Desktop for Windows.
9. Run the RStudio installer.
10. Open RStudio.

macOS

The installation process on macOS depends on your version of macOS and your Mac's processor.

1. Go to the [R Project](#).
2. Select **CRAN**.
3. Choose a CRAN mirror.
4. Select **Download R for macOS**.
5. Read the available installer descriptions carefully.
6. Download the version appropriate for your Mac.
7. Install R.
8. Go to the [RStudio Desktop download page](#).
9. Download the free version of RStudio Desktop for macOS.
10. Install RStudio.
11. Open RStudio.

macOS users

Do not guess which installer you need.

Different Mac computers may use different processors. We will verify your Mac and select the appropriate installer before continuing if you are unsure.

Check Your Installation

Once RStudio opens, locate the **Console** pane.

You should see information about the version of R that is running followed by the `>` prompt.

At the prompt, enter:

```
2 + 2
```

If R returns:

R and RStudio are communicating correctly.

Need more help?

If you have trouble installing R or RStudio, use the official installation resources:

- [R Project](#)
- [RStudio Desktop Download](#)
- [RStudio Desktop Installation Documentation](#)

If you are unsure which version to install, especially on macOS, ask for help before continuing.

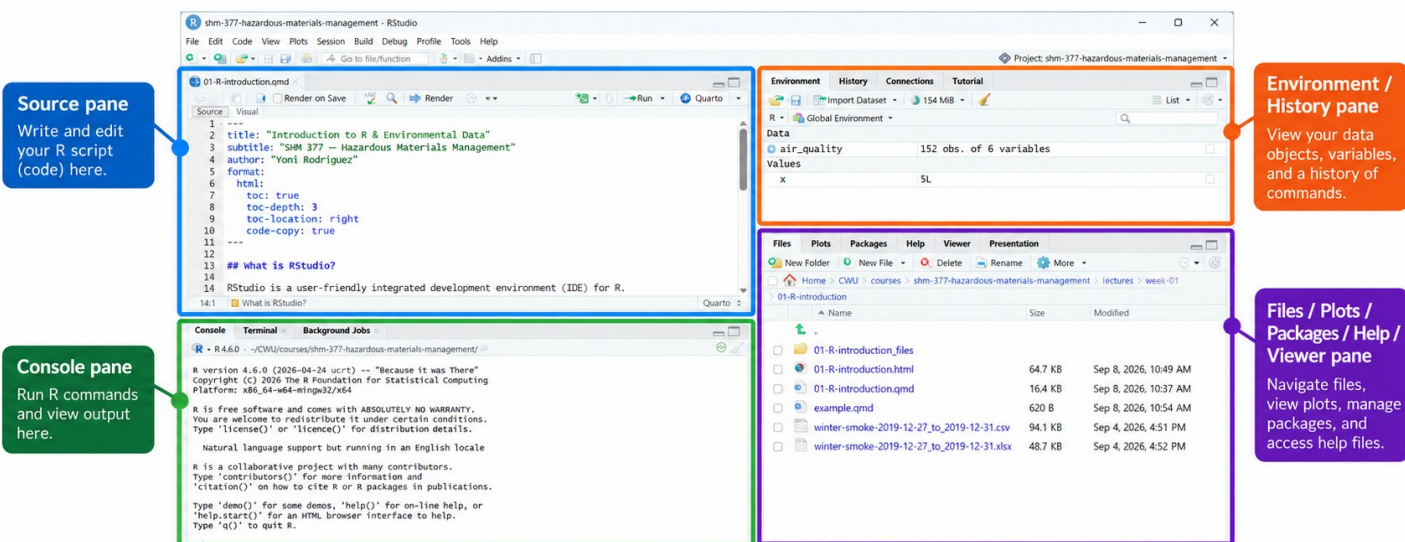
The RStudio Interface

You will commonly see four areas in RStudio:

2.2 What is RStudio?

RStudio is a user-friendly integrated development environment (IDE) for R. It makes it easier to write code, run analyses, visualize data, and manage your files and packages. The RStudio interface is divided into four main panes:

1. **Source pane (top left):** where you write and edit your R script (code).
2. **Console pane (bottom left):** where R runs your commands and displays output.
3. **Environment/History pane (top right):** shows your data objects, variables, and a history of past commands.
4. **Files/Plots/Packages/Help/Viewer pane (bottom right):** used to navigate files, view plots, manage packages, and view help files.



One important habit for this course is:

Write your commands in the Source pane rather than relying only on the Console.

The Source pane allows us to save our code and creates a record of what we did.

Organizing Your Work

Before downloading our dataset, we need to decide where our files will live.

Good file organization makes working with data much easier. Many problems in R happen because files are saved in different locations and R cannot find them.

For this course, we will keep files associated with the same project together.

Create a Course Folder

Create a folder on your computer called:

```
SHM-377
```

Inside that folder, create a folder for today's lab:

```
SHM-377/  
  virtual-lab-01/
```

We will save everything for today's analysis inside `virtual-lab-01`.

By the end of the lab, your folder may look something like this:

```
SHM-377/  
  virtual-lab-01/  
    winter-smoke-2019-12-27_to_2019-12-31.csv  
    virtual-lab-01.R
```

! Keep your project files together

Do not save the dataset in one location, your R script somewhere else, and your other project files in another folder.

Keeping related files together makes your analysis easier to reproduce and reduces problems caused by missing files or incorrect file paths.

File Naming

Use file names that tell you what the file contains.

Good:

```
winter-smoke-2019-12-27_to_2019-12-31.csv  
virtual-lab-01.R
```

Avoid names such as:

```
data.csv
data-final.csv
data-final-2.csv
stuff.csv
```

Clear file names become increasingly important as projects grow.

 A simple rule for this course

One project, one folder.

Keep the data, code, figures, and other files associated with an analysis together.

Creating Your First R Script

Now we need a place to write and save our R commands.

We will use an **R script**.

An R script is a text file that contains R commands. Instead of typing commands only into the Console and losing track of what we did, we can save our commands in a script and use them again later.

Open a New R Script

In RStudio:

1. Select **File**.
2. Select **New File**.
3. Select **R Script**.

A new blank file will open in the Source pane.

This is where we will write our R commands.

Save Your R Script

Before writing more code, save the file.

1. Select **File**.
2. Select **Save As**.
3. Navigate to your **SHM-377** folder.
4. Open the **virtual-lab-01** folder.
5. Save the file as:

```
virtual-lab-01.R
```

The `.R` file extension tells your computer that this is an R script.

Your folder should now look like this:

```
SHM-377/  
  virtual-lab-01/  
    virtual-lab-01.R
```

! Write your commands in the script

For this course, write your R commands in the R script in the Source pane. The Console shows what R is doing, but the script provides a saved record of the commands you used.

Set the Working Directory

R also needs to know where to look for the files used in our analysis.

Because we saved our R script inside `virtual-lab-01`, we can tell R to use that folder as the working directory.

In RStudio:

1. Select **Session**.
2. Select **Set Working Directory**.
3. Select **To Source File Location**.

This tells R to look for files in the same folder where the R script is saved.

💡 Why does this matter?

Later, we will ask R to open the dataset using only its file name:

```
"winter-smoke-2019-12-27_to_2019-12-31.csv"
```

R needs to know which folder to search.

Setting the working directory to the source file location tells R to begin searching inside `virtual-lab-01`.

Your First R Commands

Now that R and RStudio are installed and communicating, we can begin working in R.

We will start very simply.

A **command** is an instruction we give R.

For example:

```
2 + 2
```

```
[1] 4
```

This command asks R to perform a calculation.

R receives the instruction and returns the result underneath the code.

Try another:

```
10 / 2
```

```
[1] 5
```

At its most basic level, working with R means:

We give R an instruction and R returns something.

Creating an Object

We can also store information in R.

```
x <- 5
```

Now ask R what is stored in **x**.

```
x
```

```
[1] 5
```

The symbol:

```
<-
```

is called the assignment operator.

You can read this:

```
x <- 5
```

as:

Store the value 5 in an object called **x**.

We will use this idea constantly when working with datasets.

Getting the Dataset

Now that we have a place for our work, we need the dataset we will analyze.

For this lab, we will use:

`winter-smoke-2019-12-27_to_2019-12-31.csv`

The dataset contains ambient particle measurements collected from December 27 through December 31, 2019.

Download the Dataset

The dataset is available in the SHM 377 GitHub repository.

1. Open the SHM 377 GitHub repository.
2. Navigate to `lectures/week-01/01-R-introduction/`.
3. Select `winter-smoke-2019-12-27_to_2019-12-31.csv`.
4. Download the file.
5. Save the file inside your `virtual-lab-01` folder.

Your folder should now contain:

```
SHM-377/  
  virtual-lab-01/  
    virtual-lab-01.R  
    winter-smoke-2019-12-27_to_2019-12-31.csv
```

Now we are ready to bring the dataset into R.

Importing the Dataset

We can use the `read.csv()` function to import the file.

Function: `read.csv()`

Function name: `read.csv`

Purpose: Import data from a CSV file.

A **function** is a tool in R that performs a particular task.

The name gives us a clue about what this function does.

`read.csv()` tells R to read a CSV file.

Enter this command in your R script:

```
winter_smoke <- read.csv(  
  "winter-smoke-2019-12-27_to_2019-12-31.csv"  
)
```

Read that command from right to left.

R will:

1. find the CSV file
2. read the file
3. create a dataset in R
4. store the dataset under the name `winter_smoke`

The assignment operator `<-` is doing the same thing we saw earlier when we wrote:

```
x <- 5
```

This time, instead of storing the number 5, we are storing an entire dataset.

We now have data inside R.

Reading R Code

Before we continue, let's learn how to read some of the R code we will use.

Commands and Functions

A **command** is an instruction we give R.

For example:

```
2 + 2
```

asks R to perform a calculation.

Many of the commands we use in R involve **functions**.

A function is a tool in R that performs a particular task.

For example:

```
head()
```

The function name is `head`.

The parentheses tell R that we want to use the function.

We can place information inside the parentheses to tell the function what we want it to work with.

For example:

```
head(winter_smoke)
```

This tells R:

Use the `head()` function on the dataset called `winter_smoke`.

Throughout this course, when we introduce a new function, focus first on two questions:

1. What is the function called?
2. What does the function do?

You do not need to memorize every function.

Referring to a Variable

Our dataset is called:

```
winter_smoke
```

One variable inside that dataset is called:

```
concentration
```

In R, we can refer to that variable using:

```
winter_smoke$concentration
```

The `$` tells R to look inside `winter_smoke` for the variable called `concentration`.

You can read:

```
winter_smoke$concentration
```

as:

The `concentration` variable inside the `winter_smoke` dataset.

Later, when we write something such as:

```
mean(winter_smoke$concentration)
```

we can read it as:

Use the `mean()` function on the `concentration` variable inside the `winter_smoke` dataset.

Inspect Before You Analyze

One of the most important habits in this course will be:

Look at your data before analyzing it.

We want to know:

- Did the file import correctly?
- What variables are present?
- How many observations are there?
- What does one row represent?
- Did R interpret the variables correctly?
- Do the values look reasonable?

Look at the First Rows

The first function we will use to inspect our dataset is `head()`.

Function: `head()`

Function name: `head`

Purpose: Look at the first several rows of a dataset.

```
head(winter_smoke)
```

	date_time	concentration
1	2019-12-27 00:01:59	0.018
2	2019-12-27 00:03:59	0.019
3	2019-12-27 00:05:59	0.019
4	2019-12-27 00:07:59	0.019
5	2019-12-27 00:09:59	0.019
6	2019-12-27 00:11:59	0.019

This tells R to use the `head()` function on our `winter_smoke` dataset.

Look at the output.

Ask yourself:

What does one row appear to represent?

Look at the Last Rows

The `tail()` function allows us to look at the end of a dataset.

Function: `tail()`

Function name: `tail`

Purpose: Look at the last several rows of a dataset.

```
tail(winter_smoke)
```

		date_time	concentration
3595	2019-12-31	23:49:59	0.002
3596	2019-12-31	23:51:59	0.001
3597	2019-12-31	23:53:59	0.004
3598	2019-12-31	23:55:59	0.002
3599	2019-12-31	23:57:59	0.001
3600	2019-12-31	23:59:59	0.001

Looking at both the beginning and end of a dataset can help us confirm that the file contains the time period we expected.

Variable Names

The `names()` function shows us the names of the variables in an object.

Function: `names()`

Function name: `names`

Purpose: Show the variable names in a dataset.

```
names(winter_smoke)
```

```
[1] "date_time"      "concentration"
```

We expect to see:

```
date_time
concentration
```

Dataset Dimensions

The `dim()` function shows us the dimensions of an object.

Function: `dim()`

Function name: `dim`

Purpose: Show the number of rows and columns in a dataset.

The name `dim` is short for **dimensions**.

```
dim(winter_smoke)
```

```
[1] 3600    2
```

Inside the parentheses, we tell the function which object we want it to examine:

```
dim(winter_smoke)
```

Here:

```
dim
```

is the function name, and:

```
winter_smoke
```

is the object we are asking the function to examine.

R reports two numbers in this order:

```
rows columns
```

The rows represent observations.

The columns represent variables.

For this dataset, each row represents one monitoring measurement.

Examine the Structure

The `str()` function shows us the structure of an object.

Function: `str()`

Function name: `str`

Purpose: Examine the structure of a dataset.

The name `str` is short for **structure**.

```
str(winter_smoke)
```

```
'data.frame':   3600 obs. of  2 variables:
 $ date_time    : chr  "2019-12-27 00:01:59" "2019-12-27 00:03:59" "2019-12-27 00:05:59" "2019-12-27 00:07:59" ...
 $ concentration: num  0.018 0.019 0.019 0.019 0.019 0.019 0.019 0.019 0.019 0.019 0.019 ...
```

This tells R:

Use the `str()` function to examine the object called `winter_smoke`.

This function tells us several things at once, including:

- variable names
- variable types
- example values

This is one of the most useful functions to run immediately after importing a dataset.

Did R Understand the Date and Time?

When we examined the structure of our dataset, R may have imported the `date_time` variable as text.

If that happens, R does not yet understand that those values represent dates and times.

We need to tell R how to interpret them.

The function we will use is:

```
as.POSIXct()
```

Function: `as.POSIXct()`

Function name: `as.POSIXct`

Purpose: Convert date and time information into a format that R recognizes as date and time data.

The beginning of the function name gives us a useful clue:

```
as.
```

functions in R are often used to convert something from one type of data into another type.

For our purposes, you can think of:

```
as.POSIXct()
```

as:

Convert this information into date and time data that R can understand.

You do not need to memorize the name `as.POSIXct()`.

For now, what matters is understanding what we are asking R to do.

Our dataset is called:

```
winter_smoke
```

The date and time variable inside that dataset is called:

```
date_time
```

We can refer to that variable using:

```
winter_smoke$date_time
```

Remember that `$` tells R to look inside the dataset for a particular variable.

You can read:

```
winter_smoke$date_time
```

as:

The `date_time` variable inside the `winter_smoke` dataset.

Now we can convert that variable:

```
winter_smoke$date_time <- as.POSIXct(  
  winter_smoke$date_time  
)
```

Read the command from right to left.

First:

```
winter_smoke$date_time
```

identifies the `date_time` variable inside the `winter_smoke` dataset.

Then:

```
as.POSIXct(winter_smoke$date_time)
```

asks R to convert those values into date and time data.

Finally:

```
winter_smoke$date_time <-
```

stores the converted values back into the `date_time` variable.

Then check the structure again.

```
str(winter_smoke)
```

```
'data.frame':  3600 obs. of  2 variables:
 $ date_time      : POSIXct, format: "2019-12-27 00:01:59" "2019-12-27 00:03:59" ...
 $ concentration: num  0.018 0.019 0.019 0.019 0.019 0.019 0.019 0.019 0.019 0.019 ...
```

Compare the result with what we saw before.

Did the variable type change?

This introduces another important habit:

Do not assume your code worked. Check.

Check the Measurements

Before calculating averages or making graphs, we should inspect the range of the measurements.

Minimum

The `min()` function finds the smallest value.

Function: `min()`

Function name: `min`

Purpose: Find the minimum value.

```
min(winter_smoke$concentration)
```

```
[1] 0
```

We can read this as:

Use the `min()` function on the `concentration` variable inside the `winter_smoke` dataset.

Maximum

The `max()` function finds the largest value.

Function: `max()`

Function name: `max`

Purpose: Find the maximum value.

```
max(winter_smoke$concentration)
```

```
[1] 0.288
```

We can read this as:

Use the `max()` function on the `concentration` variable inside the `winter_smoke` dataset.

The observed concentration range in this teaching dataset is approximately:

```
0.000 to 0.288
```

At this stage, we should ask:

Do these measurements appear plausible?

Unusual Values

Suppose one concentration is much higher than most of the others.

Should we immediately delete it?

No.

An unusual value could represent:

- a real environmental event
- a nearby source
- unusual instrument behavior
- a processing problem

- something we do not yet understand

Before removing a value, ask:

- Is it physically possible?
- Do nearby observations show a similar pattern?
- Was something happening at that time?
- Could the instrument have malfunctioned?
- Do we have enough evidence to make a decision?

Unusual does not automatically mean incorrect.

Describe the Data

Now that we have inspected and checked the dataset, we can begin summarizing it.

Mean

The `mean()` function calculates the arithmetic mean.

Function: `mean()`

Function name: `mean`

Purpose: Calculate the mean of a set of values.

```
mean(winter_smoke$concentration)
```

```
[1] 0.02079694
```

We can read this as:

Use the `mean()` function on the `concentration` variable inside the `winter_smoke` dataset.

The mean gives us one way to describe the center of the measurements.

But one number cannot show us how concentrations changed through time.

Median

The `median()` function calculates the median.

Function: `median()`

Function name: `median`

Purpose: Calculate the median of a set of values.

```
median(winter_smoke$concentration)
```

```
[1] 0.018
```

The median gives us another measure of the center.

Compare the mean and median.

Are they similar?

If not, what might that suggest about the distribution of measurements?

Minimum

```
min(winter_smoke$concentration)
```

```
[1] 0
```

Maximum

```
max(winter_smoke$concentration)
```

```
[1] 0.288
```

Summary

The `summary()` function produces several summary statistics at once.

Function: `summary()`

Function name: `summary`

Purpose: Produce a summary of an object.

```
summary(winter_smoke$concentration)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
0.0000	0.0130	0.0180	0.0208	0.0300	0.2880

For a numeric variable, R provides several descriptive statistics.

Look at the output.

What information does R provide?

What information is still missing?

Descriptive Statistics Have Limits

A numerical summary can tell us about:

- the center
- the range
- the distribution

But it cannot tell us when concentrations changed.

For that, we need to look at the measurements through time.

Visualizing Concentration Through Time

Because these measurements were repeatedly collected over several days, a time series graph is a natural first visualization.

We will place:

time on the x axis

and

particle concentration on the y axis

These measurements were collected using a TSI DustTrak.

The particle concentration measurements are reported in:

milligrams per cubic meter (mg/m^3)

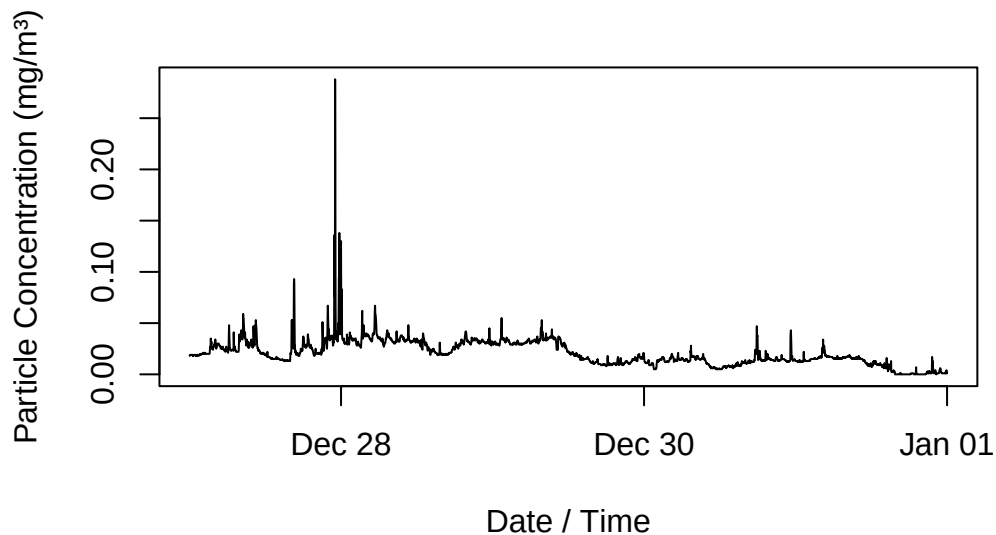
The function we will use is `plot()`.

Function: `plot()`

Function name: `plot`

Purpose: Create a graph.

```
plot(
  winter_smoke$date_time,
  winter_smoke$concentration,
  type = "l",
  xlab = "Date / Time",
  ylab = "Particle Concentration ( $\text{mg}/\text{m}^3$ )"
)
```



In this lab document, the graph appears directly underneath the code.

When you run the command from your R script in RStudio, the graph will appear in the **Plots** pane.

Reading the Plot Function

The `plot()` function is more complicated than the functions we have used so far.

That makes it a useful example for learning how to read R code.

Look at the command again:

```
plot(  
  winter_smoke$date_time,  
  winter_smoke$concentration,  
  type = "l",  
  xlab = "Date / Time",  
  ylab = "Particle Concentration (mg/m³)"  
)
```

Everything inside the parentheses provides information to the `plot()` function.

These pieces of information are called **arguments**.

Arguments

An **argument** gives a function information it needs to perform its task.

Our `plot()` function contains five arguments:

```
winter_smoke$date_time  
winter_smoke$concentration  
type = "l"  
xlab = "Date / Time"  
ylab = "Particle Concentration (mg/m³)"
```

Each argument is separated by a comma.

Why Do We Use Commas?

Inside a function, commas separate one argument from the next.

For example:

```
plot(  
  winter_smoke$date_time,  
  winter_smoke$concentration  
)
```

The comma tells R that:

```
winter_smoke$date_time
```

is one argument and:

```
winter_smoke$concentration
```

is another argument.

We do not use a semicolon here.

A semicolon can be used in R to separate complete commands or expressions.

For example:

```
x <- 5; x
```

contains two complete instructions.

That is different from separating arguments inside a function.

For this course, use **commas to separate arguments inside functions**.

Why Does `date_time` Come First?

The first two arguments in our `plot()` function are:

```
winter_smoke$date_time,  
winter_smoke$concentration
```

The `plot()` function expects information about what should appear on the x axis and what should appear on the y axis.

When we provide arguments without writing their names, R uses their position.

The first argument is used for the x axis.

The second argument is used for the y axis.

Therefore:

```
winter_smoke$date_time
```

comes first because we want time on the x axis.

And:

```
winter_smoke$concentration
```

comes second because we want particle concentration on the y axis.

You can read:

```
plot(  
  winter_smoke$date_time,  
  winter_smoke$concentration  
)
```

as:

Plot `date_time` on the x axis and `concentration` on the y axis.

Positional Arguments

The first two arguments are being identified by their position.

These are called **positional arguments**.

We could make their meaning more explicit by writing:

```
plot(  
  x = winter_smoke$date_time,  
  y = winter_smoke$concentration  
)
```

This tells R exactly the same thing.

Here:

```
x =
```

identifies the x axis data.

And:

```
y =
```

identifies the y axis data.

For our first graph, we are using the shorter form:

```
plot(  
  winter_smoke$date_time,  
  winter_smoke$concentration  
)
```

Both forms tell R to use `date_time` for x and `concentration` for y.

Named Arguments

The remaining arguments are written differently:

```
type = "l",  
xlab = "Date / Time",  
ylab = "Particle Concentration (mg/m3)"
```

These are **named arguments**.

Instead of relying on position, we explicitly tell R which setting we are changing.

What Does `type = "l"` Mean?

The `type` argument controls how the observations are drawn.

```
type = "l"
```

The letter:

```
l
```

stands for **line**.

This tells R to connect the observations with a line.

The quotation marks tell R that "l" is text rather than the name of an object.

What Does xlab Mean?

```
xlab = "Date / Time"
```

xlab means **x axis label**.

This tells R to display:

```
Date / Time
```

as the label for the x axis.

What Does ylab Mean?

```
ylab = "Particle Concentration (mg/m3)"
```

ylab means **y axis label**.

This tells R to display:

```
Particle Concentration (mg/m3)
```

as the label for the y axis.

The unit is part of the label because the numbers alone do not tell us what was measured or how the measurement should be interpreted.

What Does = Mean Inside the Function?

Earlier we used:

```
<-
```

to store information in an object.

For example:

```
x <- 5
```

Inside a function, we will often see:

```
=
```

used to provide a value for a named argument.

For example:

```
type = "l"
```

means:

Use "l" for the `type` argument.

And:

```
xlab = "Date / Time"
```

means:

Use "Date / Time" for the `xlab` argument.

The `=` here is helping us specify how the function should behave.

Read the Entire Command

We can now read the complete function:

```
plot(  
  winter_smoke$date_time,  
  winter_smoke$concentration,  
  type = "l",  
  xlab = "Date / Time",  
  ylab = "Particle Concentration (mg/m³)"  
)
```

as:

Use the `plot()` function. Put `date_time` on the x axis. Put `concentration` on the y axis. Connect the observations with a line. Label the x axis "Date / Time". Label the y axis "Particle Concentration (mg/m³)".

That is all the command is asking R to do.

Why Units Matter

A number without a unit can be difficult or impossible to interpret correctly.

In this dataset, the concentration values came from a TSI DustTrak and are reported in:

milligrams per cubic meter (mg/m³)

However, notice that our CSV variable is simply called:

```
concentration
```

The variable name itself does not tell us the unit.

This is common when working with environmental monitoring data.

A CSV file may contain the numerical measurements but may not contain all of the information needed to understand those measurements.

That information may instead come from:

- instrument settings
- field notes
- data dictionaries
- project documentation
- sampling protocols
- laboratory reports
- instrument manuals
- knowledge of the monitoring method

This means that importing a dataset is not enough.

We also need to understand:

What was measured, how was it measured, and what units were used?

The appropriate unit depends on the agent, the environmental medium, the instrument, and the measurement method.

For example, an air concentration may be expressed differently from a concentration measured in water or soil.

Different agents may also be reported using different units.

Understanding the standard measurement practices for the **medium and agent being evaluated** is part of understanding the dataset.

! Never assume the units

Do not infer a unit only from the numerical values in a dataset.

Confirm the units using the instrument information, project documentation, sampling method, or another reliable source.

If the unit cannot be verified, do not present one as fact.

For this dataset, we have verified the measurement information, so our graph should identify the y axis as:

Particle Concentration (mg/m^3)

Including the unit makes the graph more informative and preserves important measurement context.

Look Before Explaining

Study the graph before trying to explain it.

Ask:

- Is concentration constant?
- Are there peaks?
- When do the largest peaks occur?
- Are there periods of relatively low concentration?
- Are some periods more variable?
- Do any patterns appear to repeat?

The first step is observation.

Observation and Interpretation

A statement such as:

Particle concentration varied throughout the monitoring period, with periods of relatively low concentration interrupted by higher measured concentrations.

is an observation based on the dataset.

A statement such as:

Residential wood burning caused the peaks.

is a source attribution claim.

The dataset alone does not prove that.

What Can We Conclude?

From the measurements we can examine:

- when concentrations increased
- how high measured concentrations became
- how concentrations changed through time
- whether elevated periods persisted
- how variable the measurements were

What Can We Not Conclude?

From this dataset alone, we cannot determine:

- the exact source of every particle
- why every concentration increase occurred
- whether one source caused a particular peak

Additional information would be needed.

What Other Data Could Help?

The monitoring results may lead us to new questions.

For example:

- What was the wind speed?
- What was the wind direction?
- What was the temperature?
- Were there atmospheric inversions?
- Were there nearby emission sources?
- Did local heating activity change?

This is how data analysis often works.

One answer produces better questions.

Return to the Original Question

Our original question was:

What happened to ambient particle concentrations during five days of winter monitoring?

We now have several ways to answer that question.

We:

- inspected the dataset
- checked its structure
- examined its measurement range
- calculated descriptive statistics
- plotted concentration through time
- interpreted the observed pattern

The SHM 377 Data Workflow

Throughout this course, we will repeatedly use a workflow like this:

```
Question
  ↓
Data
  ↓
Check
  ↓
Describe / Analyze
  ↓
Visualize
  ↓
Interpret
  ↓
Communicate / Manage
```

The specific commands will change.

The reasoning process will remain similar.

Connecting Data to Hazardous Materials Management

In hazardous materials management, we may begin with questions about:

```
Material
  ↓
Properties
  ↓
Source / Activity
  ↓
Release
  ↓
Environmental Medium
  ↓
Transport
  ↓
```

```
Exposure
  ↓
Dose
  ↓
Effects
```

Monitoring provides evidence about parts of that system.

Data analysis helps us move from measurements toward evaluation and decision making.

```
Hazardous Materials Problem
  ↓
Measurement
  ↓
Data
  ↓
Evaluation
  ↓
Interpretation
  ↓
Management / Control
```

Reproducibility

One reason we are using R is that the analysis itself can be saved.

For example:

```
winter_smoke <- read.csv(
  "winter-smoke-2019-12-27_to_2019-12-31.csv"
)

head(winter_smoke)

summary(winter_smoke$concentration)

plot(
  winter_smoke$date_time,
  winter_smoke$concentration,
  type = "l",
  xlab = "Date / Time",
  ylab = "Particle Concentration (mg/m³)"
)
```

These commands provide a record of what we did.

Someone can inspect the code and understand how the analysis was produced.

Later, this idea will lead naturally into Quarto reports.

You Do Not Need to Memorize Everything

Today you encountered commands such as:

```
read.csv()
head()
tail()
names()
dim()
str()
as.POSIXct()
min()
max()
mean()
median()
summary()
plot()
```

You do not need to memorize all of these immediately.

What matters more is understanding why we use them.

Ask yourself:

What is my question?

What does my dataset contain?

Did the data import correctly?

Do the measurements look reasonable?

How can I describe them?

How can I visualize them?

What does the evidence show?

What can I not conclude?

What You Accomplished

In this first virtual lab, you:

- worked with a real environmental monitoring dataset
- imported data into R
- inspected the dataset
- checked the measurements
- calculated basic descriptive statistics
- created a time series visualization
- interpreted the results
- distinguished measurement from source attribution

The Main Idea

We are not learning R for the sake of learning R.

We are learning how to use data to investigate, evaluate, and communicate hazardous materials problems.

Where We Go Next

We will continue building on the same workflow throughout SHM 377.

Over time, we will move from basic R commands toward:

```
R
↓
R scripts
↓
data analysis
↓
visualization
↓
interpretation
↓
reproducible Quarto reports
```

We do not need to learn everything at once.

We start with the question.

Then we use the data to work toward an answer.